# Übungsblatt 4: Funktionen & der Stack

**Mikrocomputertechnik 1**

**Musterlösung**

## Zielsetzung

In diesem Übungsblatt verlassen Sie das Spaghetti-Code-Niveau und strukturieren Ihre Programme in saubere, wiederverwendbare Unterprogramme (Funktionen). Sie lernen, wie Sie Argumente übergeben (`a0`–`a7`), den dynamischen Stack für lokale Backups (Prolog/Epilog) nutzen und wie Sie den gefährlichen Register-Krieg zwischen Caller und Callee mithilfe der ABI-Regeln verhindern.

## Aufgabe 1: Die ABI und der Register-Krieg

Ein solides Verständnis der ABI (Application Binary Interface) schützt Sie vor stundenlangem Debugging. Beantworten Sie die folgenden Fragen.

### Caller-Saved nach `jal`

Ein Hauptprogramm (Caller) speichert einen wichtigen Zähler im Register `t0` und ruft dann eine externe Funktion `Print()` mit dem Befehl `jal` auf. Darf der Aufrufer davon ausgehen, dass `t0` nach der Rückkehr von `Print()` noch denselben Wert hat? Warum oder warum nicht?

### Callee-Saved und Rettungspflicht

Die Funktion `Print()` aus der vorherigen Teilaufgabe möchte intern das Register `s0` für eine eigene Rechnung nutzen. Welche Pflicht hat `Print()` gegenüber dem Aufrufer, bevor sie `s0` überschreibt?

### Ihre Lösung

*(Notieren Sie Ihre Antworten hier.)*

**Musterlösung Aufgabe 1**

Caller-Saved nach `jal`:

- Nein, der Aufrufer darf nicht davon ausgehen.
- `t0` ist laut ABI ein Caller-Saved (flüchtiges) Register.

- `Print()` darf es ohne Vorwarnung überschreiben.
- Braucht der Caller den Wert nach dem Aufruf noch, muss er ihn vor `jal` selbst auf dem Stack sichern.

Callee-Saved und Rettungspflicht:

- `s0` ist ein Callee-Saved (sicheres) Register.
- Der Aufrufer verlässt sich darauf, dass es unangetastet bleibt.
- Will `Print()` `s0` nutzen, muss sie den alten Wert im Prolog auf den Stack retten und ihn im Epilog (vor `ret`) wiederherstellen.

## Aufgabe 2: Stack-Management und der `sp`-Pointer

Der Stack wächst im Arbeitsspeicher dynamisch nach unten (zu kleineren Adressen).

### Stack-Reservierung

Sie müssen in einer Funktion Platz für exakt drei 32-Bit-Werte (z. B. `ra`, `s0`, `s1`) auf dem Stack reservieren. Wie lautet der Assembler-Befehl, um den Stack Pointer (`sp`) korrekt zu verschieben?

### Off-by-One im Epilog

Beschreiben Sie die Gefahr eines Off-by-One-Fehlers im Epilog. Was passiert, wenn Sie im Prolog den `sp` um $-12$ verschieben, im Epilog aber versehentlich nur um $+8$ korrigieren, bevor Sie `ret` aufrufen?

### Ihre Lösung

*(Notieren Sie Ihre Antworten hier.)*

**Musterlösung Aufgabe 2**

Stack-Reservierung:

- Drei 32-Bit-Wörter entsprechen 12 Bytes ($3 \times 4$).
- Da der Stack nach unten wächst, muss der Pointer verkleinert werden.
- Befehl: `addi sp, sp, -16`

Off-by-One im Epilog:

- Das verletzt das Gleichgewichts-Gesetz des Stacks.
- Nach der Rückkehr steht `sp` 4 Bytes tiefer als vor dem Aufruf.
- Relative Offsets des Callers verschieben sich; spätere `lw`-Befehle (z. B. `lw s0, 0(sp)`) lesen falsche Daten.
- Das Programm wird fehlerhaft oder abstürzen.

## Aufgabe 3: Venus — Die Leaf-Funktion

Schreiben Sie ein Hauptprogramm `main` und eine einfache Leaf-Hilfsfunktion (eine Funktion, die keine weiteren Funktionen aufruft).

### Aufgabenstellung

Wir wollen eine Zahl verdoppeln. Da RV32I keinen `mul`-Befehl hat, schreiben wir eine Funktion `Verdopple`.

1. Im `main`: Laden Sie die Konstante 15 in `a0`.
2. Rufen Sie `Verdopple` mit `jal` auf.
3. `Verdopple` verdoppelt den übergebenen Wert (ALU-Befehl `slli` aus Labor 2) und speichert das Ergebnis im Rückgaberegister `a0`.
4. Die Funktion kehrt mit `ret` zurück.
5. Im `main`: Geben Sie das Ergebnis über `ecall` aus (Venus-ABI: Dienstcode 1 in `a0`, Wert in `a1`) und beenden Sie das Programm sauber (Dienstcode 10).

Hinweis: Als Leaf-Funktion ohne `s`-Register brauchen Sie hier noch keinen Prolog/Epilog. Achten Sie darauf, den Rückgabewert vor dem Print-`ecall` nach `a1` zu kopieren — sonst überschreibt der Dienstcode in `a0` das Ergebnis.

### Ihre Lösung

*(Fügen Sie Ihren Assembler-Code hier ein.)*

**Musterlösung Aufgabe 3**

```
.text
.globl main

main:
    # Parameter für die Funktion vorbereiten
    li a0, 15

    # Funktionsaufruf (Jump and Link)
    jal ra, Verdopple

    # Rückgabewert (30) liegt in a0 - vor Print nach a1 sichern
    mv a1, a0
    li a0, 1        # Venus: Print Integer
    ecall

    # Programm sauber beenden
    li a0, 10       # Venus: Exit
    ecall
```

```
# --- Unterprogramm (Leaf) ---
Verdopple:
    slli a0, a0, 1  # a0 = a0 * 2
    ret
```

## Aufgabe 4: Venus — Nested Calls (Verschachtelung)

Wir erweitern das Programm zu einer Non-Leaf-Funktion, die selbst einen weiteren Aufruf tätigt. Dazu brauchen Sie zwingend einen Stack-Frame, um die Rücksprungadresse (`ra`) zu sichern.

### Aufgabenstellung

Schreiben Sie eine Funktion `QuadratPlusEins`, die den Wert in `a0` mit sich selbst multipliziert und danach `AddiereEins` aufruft (Ergebnis um 1 erhöhen).

1. Prolog in `QuadratPlusEins`: Reservieren Sie Platz auf dem Stack und speichern Sie `ra`.
2. Body: Führen Sie die Rechnung aus (`mul a0, a0, a0`; Venus mit M-Extension) und rufen Sie danach `jal ra, AddiereEins` auf.
3. Epilog: Stellen Sie `ra` aus dem Stack wieder her, geben Sie den Stack-Platz frei und rufen Sie `ret` auf.
4. Testen Sie aus `main` mit dem Step-Debugger und beobachten Sie Register `ra`.

### Ihre Lösung

*(Fügen Sie Ihren Assembler-Code hier ein.)*

**Musterlösung Aufgabe 4**

```
.text
.globl main

main:
    li a0, 5
    jal ra, QuadratPlusEins

    # Ergebnis (26) drucken
    mv a1, a0
    li a0, 1
    ecall

    li a0, 10
    ecall

# --- Funktion 1 (Non-Leaf) ---
QuadratPlusEins:
    # --- PROLOG ---
```

```
    addi sp, sp, -16
    sw ra, 12(sp)

    # --- BODY ---
    mul a0, a0, a0     # a0 = a0 * a0 (M-Extension)
    jal ra, AddiereEins

    # --- EPILOG ---
    lw ra, 12(sp)
    addi sp, sp, 16
    ret

# --- Funktion 2 (Leaf) ---
AddiereEins:
    addi a0, a0, 1
    ret
```